

QA-2.2: Linguagens de Programação:

Questões de aula (QA) para a aula 2 de

"Sintaxe e Semântica"

Total de pontos **1/4** ?

Estas questões devem ser respondidas após o envio das questões QP, feitas em preparação para a aula 2 de "Introdução". Essas questões só estarão disponíveis no dia da respectiva aula e para serem respondidas em até 4 (quatro) dias OU até o dia da próxima aula (o que vier antes).

Idealmente, neste tempo você poderá:

- * Responder individualmente as questões
- * Discutir com o seu colega de classe as respostas escolhidas.
- * Resubmeter a sua resposta.

A sua nota será a proporção dos acertos das questões. Por isso, responda com cuidado e tempo.

As suas respostas só serão consideradas se submetidas até esse prazo de quatro dias. Depois desse prazo, a atividade será considerada não entregue e a sua nota será automaticamente zero.

E-mail *

ramonsoares@discente.ufcat.edu.br

0 de 0 pontos

Nome *

Para registro das respostas

RAMON SOARES MENDES DE MENESES LEITE

Tipo de Submissão de QA *

- ☒ Primeira submissão (individual)
- ☐ Segunda submissão (discutida com outro aluno)

Identificação de par em discussão

0 de 0 pontos

Procure na descrição da aula (Google Classroom) qual é a pessoa alocada para discutir com você essa questão.

Aluno com quem a questão foi discutida *

- ☐ ANA CLARA ALVES ARAÚJO
- ☐ ANDRE LUIZ DE PAULA MACHADO
- ☐ ANDRE PARANHOS CHAUD
- ☐ ANDRE TONI FOGUEL
- ☐ BRUNO GOMES
- ☐ DANILO BORGES DUTRA
- ☐ EDUARDO MELLO SAMPAIO BERNARDINO
- ☐ EZEQUIEL PIRES E SILVA
- ☐ FILIPE ANDRADE PERES
- ☐ GABRIEL ARRUDA PARANHOS NETTO
- ☐ GABRIELA FELIX DE SOUZA
- ☐ GIOVANNI DE OLIVEIRA FELISBERTO
- ☐ IGOR ALVES RIBEIRO
- ☐ ITALO MATOS OLIVEIRA
- ☐ JOAO PEDRO PORTILHO SILVA
- ☐ KAIO BORGES NASCIMENTO
- ☐ KALIL AZIZ SANTOS CHAMI
- ☐ LUCAS ELIAS DE ANDRADE CRUVINEL
- ☐ MATEUS PEREIRA DA SILVA
- ☐ RAMON SOARES MENDES DE MENESES LEITE
- ☐ WELESON BATISTA JACINTO

1 de 1 pontos

QA-2.2.1 Considere a seguinte gramática de atributos da Figura 1 abaixo.

Considere decorar a árvore de gramática de atributos para a última linha do código abaixo, considerando que C é real.

A = 12
B = 1.2
C = A + B

Figura 1: Gramática de Atributos

```
1) 1.1 Regra Sintática: <var> -> A | B | C
    1.2 Regra Semântica: <var>.actual_type <- lookup(<var>.string)
2) 2.1 Regra Sintática: <assign> -> <var> = <expr>
    2.2 Regra Semântica: <expr>.expected_type <- <var>.actual_type
3) 3.1 Regra Sintática: <expr> -> <var>
    3.2 Regra Semântica: <expr>.actual_type <- <var>.actual_type
    3.3 Predicado: <expr>.actual_type == <expr>.expected_type
4) 4.1 Regra Sintática: <expr> -> <var>[2] + <var>[3]
    4.2 Regra Semântica: <expr>.actual_type <-
      if (<var>[2].actual_type == int) and (<var>[3].a_t == int)
        return int
      else return real
    4.3 Predicado: <expr>.actual_type == <expr>.expected_type
```

✓ QA-2.2.1: Após a decoração qual será o valor de actual_type e expected_type para <expr>, respectivamente? *

1/1

- ☐ int e real
- ☐ real e int
- ☒ real e real
- ☐ int e int

0 de 1 pontos

QA-2.2.2 Considere as modificações indicadas na Figura 2 na gramática de atributos (incluindo a regra sintática) da questão anterior (Figura 1), para permitir a soma de quaisquer três variáveis A, B e C.

Figura 1: Gramática da questão anterior

```
1) 1.1 Regra Sintática: <var> -> A | B | C
    1.2 Regra Semântica: <var>.actual_type <- lookup(<var>.string)
2) 2.1 Regra Sintática: <assign> -> <var> = <expr>
    2.2 Regra Semântica: <expr>.expected_type <- <var>.actual_type
3) 3.1 Regra Sintática: <expr> -> <var>
    3.2 Regra Semântica: <expr>.actual_type <- <var>.actual_type
    3.3 Predicado: <expr>.actual_type == <expr>.expected_type
4) 4.1 Regra Sintática: <expr> -> <var>[2] + <var>[3]
    4.2 Regra Semântica: <expr>.actual_type <-
      if (<var>[2].actual_type == int) and (<var>[3].a_t == int)
        return int
      else return real
    4.3 Predicado: <expr>.actual_type == <expr>.expected_type
```

Figura 2: Modificações propostas à gramática

```
/* MODIFICAÇÃO 1 */
4.1 Regra Sintática: <expr> -> <var>[2] + <expr>[3] + <var>[4]
4.2 Regra Semântica: <expr>.actual_type <-
  if (<var>[2].actual_type == int) and (<expr>[3].a_t == int)
    and (<var>[4].actual_type == int)
    return int
  else return real

/* MODIFICAÇÃO 2 */
4.1 Regra Sintática: <expr> -> <var>[2] + <expr>[3]
4.2 Regra Semântica: <expr>.actual_type <-
  if (<var>[2].actual_type == int) and (<expr>[3].a_t == int)
    return int
  else return real

/* MODIFICAÇÃO 3 */
5) 5.1 Regra Sintática: <expr> -> <var>[2] + <expr>
```

✗ QA-2.2.2: Quais das propostas de modificações são consistentes com o objetivo a ser atingido? *

0/1

- ☐ NENHUMA das regras propostas é suficiente para descrever.
- ☐ Modificação 1
- ☐ Modificação 2
- ☐ TODAS as regras propostas são suficientes.
- ☒ Modificação 3

Resposta correta

- ☒ Modificação 2

Feedback

A "Modificação 1" não permite que operações com dois operandos, enquanto a "Modificação 2" permite.

0 de 1 pontos

QA-2.2.3 Considere as modificações da Figura 2 a serem aplicadas na gramática de atributos anterior (Figura 1) para permitir a concatenação entre strings, usando o mesmo operador "+".

Figura 1: Gramática de atributos original

```
1) 1.1 Regra Sintática: <var> -> A | B | C
    1.2 Regra Semântica: <var>.actual_type <- lookup(<var>.string)
2) 2.1 Regra Sintática: <assign> -> <var> = <expr>
    2.2 Regra Semântica: <expr>.expected_type <- <var>.actual_type
3) 3.1 Regra Sintática: <expr> -> <var>
    3.2 Regra Semântica: <expr>.actual_type <- <var>.actual_type
    3.3 Predicado: <expr>.actual_type == <expr>.expected_type
4) 4.1 Regra Sintática: <expr> -> <var>[2] + <var>[3]
    4.2 Regra Semântica: <expr>.actual_type <-
      if (<var>[2].actual_type == int) and (<var>[3].a_t == int)
        return int
      else return real
    4.3 Predicado: <expr>.actual_type == <expr>.expected_type
```

Figura 2: Modificações propostas para permitir uso do operador "+" para concatenação. O símbolo <=> significa "implica".

```
/* MODIFICAÇÃO 1 */
4.1 Regra Sintática: <expr> -> <var>[2] + <expr>[3]
4.2 Regra Semântica: <expr>.actual_type <-
  if (<var>[2].actual_type == string)
    return string
  else if (<var>[2].actual_type == int) and (<expr>[3].a_t == int)
    return int
  else return real
4.3 Predicado: (<var>[2].actual_type == string) <=> (<expr>[3].a_t == string)

/* MODIFICAÇÃO 2 */
4.1 Regra Sintática: <expr> -> <var>[2] + <expr>[3]
4.2 Regra Semântica: <expr>.actual_type <-
  if (<var>[2].actual_type == string) and (<expr>[3].a_t == string)
    return string
  else if (<var>[2].actual_type == int) and (<expr>[3].a_t == int)
    return int
  else return real
4.3 Predicado: (<var>[2].actual_type == string) <=> (<expr>[3].a_t == string)

/* MODIFICAÇÃO 3 */
4.1 Regra Sintática: <expr> -> <var>[2] + <expr>[3]
4.2 Regra Semântica: <expr>.actual_type <-
  if (<var>[2].actual_type == string) and (<expr>[3].a_t == string)
    return string
  else if (<var>[2].actual_type == int) and (<expr>[3].a_t == int)
    return int
  else return real

/* MODIFICAÇÃO 4 */
4.1 Regra Sintática: <expr> -> <var>[2] + <var>[3]
4.2 Regra Semântica: <expr>.actual_type <-
  if (<var>[2].actual_type == string) or (<expr>[3].a_t == string)
    return string
  else if (<var>[2].actual_type == int) and (<expr>[3].a_t == int)
    return int
  else return real
```

✗ QA-2.2.3: Quais modificações são consistentes para atingir o requisito abaixo? *

0/1

- ☒ Apenas a modificação 2
- ☐ Nenhuma das modificações atende ao requisito.
- ☐ Quaisquer modificações são válidas.
- ☐ Apenas a modificação 1
- ☐ Apenas as modificações 1 ou 2
- ☐ Apenas as modificações 1, 2 ou 3

Resposta correta

- ☒ Apenas as modificações 1 ou 2

0 de 1 pontos

QA-2.2.4 Considere as instruções da Figura 1 para um certa máquina virtual.

Especifique a semântica operacional do switch/case do C, utilizando as instruções da Figura 1. Caso necessário, a BNF do switch-case do C encontra-se descrita na Figura 2.

Considere o código abaixo, que faz o uso do switch/case de C e para o qual se deverá descrever a semântica operacional.

```
switch (a) {
  case 1:
    statement-case1;
    break;
  case 2:
    statement-case2;
  default:
    statement-default;
}
```

Figura 1: Instruções da máquina virtual e significado

```
label: // nomeia ponto do código
if (expr) goto label; // lista qualquer de comandos
goto label // vá para trecho do código
```

Figura 2: BNF do switch/case de C, caso necessário.

```
<selection-statement> ::= if ( <expression> ) <statement>
                        | if ( <expression> ) <statement> else <statement>
                        | switch ( <expression> ) <statement>

<statement> ::= <labeled-statement>
              | <expression-statement>
              | <compound-statement>
              | <selection-statement>
              | <iteration-statement>
              | <jump-statement>

<labeled-statement> ::= <identifier> : <statement>
                    | case <constant-expression> : <statement>
                    | default : <statement>

<jump-statement> ::= goto <identifier> ;
                  | continue ;
                  | break ;
                  | return <expression>? ;
```

✗ QA-2.2.4: Quais das semânticas operacionais propostas são consistentes com o código a ser especificado? *

0/1

- ☐ Opção 2
- ☐ Opção 1
- ☐ Opção 3
- ☒ Opção 4

Resposta correta

- ☒ Opção 1